

Placement Portal Application — V2

Project Report | Modern Application Development II | Jan 2026

1. Student Details

Name: Krishna J

Roll Number: 23F2001849

Email: 23f2001849@ds.study.iitm.ac.in

Programme: BS Degree in Data Science and Applications, IIT Madras

2. Project Description and Approach

- **The problem statement:** placement coordination in institutes runs across emails, spreadsheets, and phone calls. Companies post drives with no visibility on who applied. Students apply to drives they are not eligible for. The admin tracks everything manually. This creates delays, missed deadlines, and data errors.
- **My approach** was to split the problem into three role-based dashboards. The admin approves company registrations, approves placement drives, manages students, blacklists bad actors, and tracks overall statistics. Companies register, wait for admin approval, then create drives and manage applications using a status state machine: applied → shortlisted → selected or rejected. Students browse approved drives filtered by their eligibility, apply, track history, and download their application records as CSV.
- **Backend DB,** I built the data layer first. All 8 SQLAlchemy models were written before any route or Vue component was touched. This forced clarity on relationships early — for example, the Application table has a composite UniqueConstraint on (student_id, drive_id), so the database rejects duplicates even if the route-level check has a race condition. After models, I wrote all Flask routes and tested each one through Swagger before writing any frontend code.
- **For background jobs,** Celery with Redis as the broker handles three tasks: a daily reminder that sends Google Chat messages to students whose drives have deadlines within 2 days, a monthly placement report emailed to admin on the 1st of each month, and user-triggered CSV exports where the frontend polls for completion status. Redis also caches the drives list and dashboard stats with 60-second TTL and invalidation on writes.
- **The frontend** is a Vue 3 SPA built with Vite. Vue Router guards check role before allowing navigation. The Pinia auth store holds the JWT tokens and user role, rehydrating from localStorage on page refresh. Axios has a response interceptor that silently refreshes the access token when it gets a 401, then retries the original request.

3. AI/LLM Declaration

Tool used: Claude (Anthropic), primarily for debugging support, syntax clarification, and resolving implementation errors encountered during development.

What I built without AI help: all 8 SQLAlchemy model files (schema design decisions, relationship types, constraints), the JWT two-token auth strategy, the admin routes (first role, sets the pattern for the others), the state machine dict in the application status route, debugging all runtime errors by reading tracebacks.

Where AI was involved: AI assistance was used to interpret traceback messages, identify likely causes of runtime issues, clarify framework-specific syntax, and suggest corrections during debugging. All suggested changes were reviewed, tested, and adjusted before being incorporated into the project.

I am able to explain the project structure, implementation logic, and code used in the submitted application.

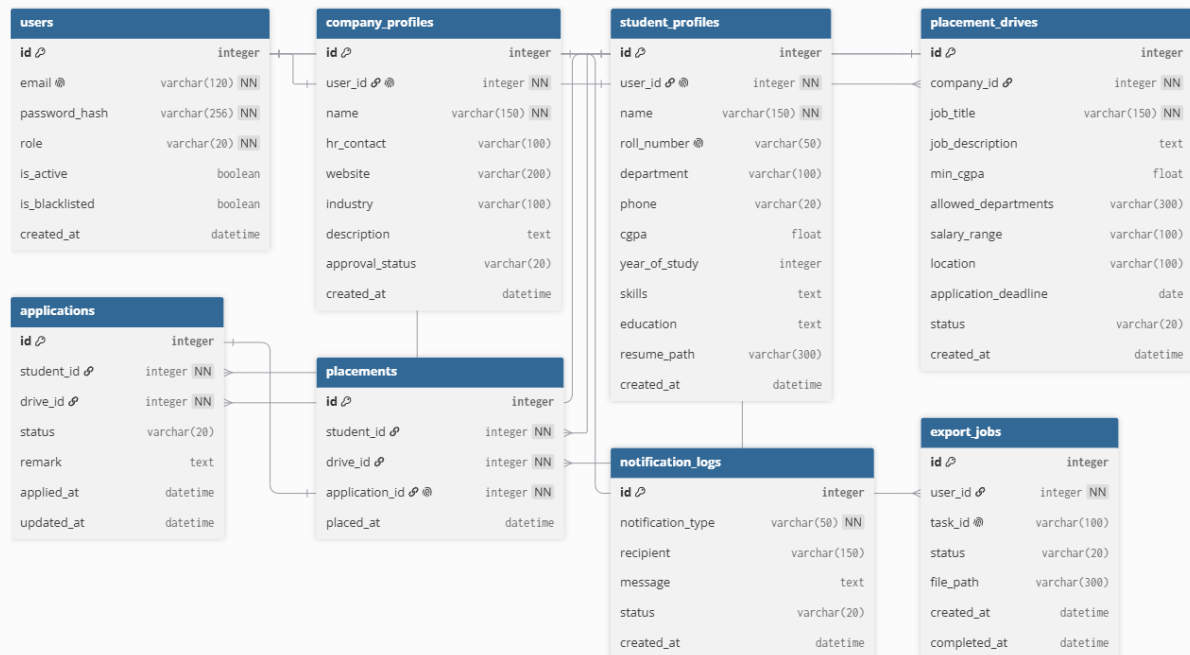
Component	% of Project	% AI Use	Notes
Backend (Flask + SQLAlchemy)	30%	12%	Models hand-coded, routes scaffolded
Frontend (Vue + HTML)	40%	25%	Admin hand-coded, others scaffolded
Styling / CSS	8%	8%	Bootstrap classes + custom style.css
Auth + CRUD Logic	15%	5%	JWT strategy mine, interceptor scaffolded
Extras (Celery, Redis)	7%	5%	First task each hand-coded, rest scaffolded
Weighted Overall	100%	~15%	All code read and understood before use

4. Frameworks and Libraries Used

Framework / Library	Purpose
Flask 3.0	Web framework, REST API, serves Vue production build
Flask-JWT-Extended 4.7	JWT auth — access token (15 min), refresh token (7 days)
Flask-SQLAlchemy 3.1 + SQLAlchemy 2.0	ORM — 8 models, relationships, SQLite
Flask-Mail 0.10	SMTP email for monthly reports and CSV export emails
Flask-Caching 2.3	Redis cache integration
Flasgger 0.9	Swagger UI at /apidocs from api.yaml
Celery 5.4 + Celery Beat	Background job queue, scheduled tasks
redis-py 5.2	Python client for Redis (broker, cache)
Werkzeug	Password hashing via generate/check_password_hash
python-dotenv	Loads .env for credentials
Vue 3 (Vite + script setup)	Frontend SPA, Composition API
Vue Router 4	SPA routing with beforeEach role guards
Pinia	Auth store and flash message store
Axios	API client with JWT interceptor and auto-refresh
Bootstrap 5	Styling and layout
SQLite	Database — created programmatically via db.create_all()
Redis (Memurai on Windows)	Celery broker/backend (DB 1/2), app cache (DB 3)
Google Chat Webhook	Notification channel for reminders and export alerts

5. ER Diagram

Generated from dbdiagram.io using the DBML code for all 8 tables. See attached image below.



6. API Resource Endpoints

Total: 35 endpoints across 5 route groups. All JSON. Auth via Authorization: Bearer <token> header.

Method	Endpoint	Role	Purpose
POST	/api/register/student	Public	Student self-registration
POST	/api/register/company	Public	Company registration (pending approval)
POST	/api/login	Public	Login, returns access + refresh tokens
POST	/api/refresh	Auth	Get new access token via refresh token
POST	/api/logout	Auth	Logout (clears client token)
GET	/api/me	Auth	Get current user info
GET	/api/admin/dashboard	Admin	Counts: students, companies, drives, apps
GET	/api/admin/companies	Admin	List companies, filter by status
PUT	/api/admin/companies/<id>/approve	Admin	Approve a company
PUT	/api/admin/companies/<id>/reject	Admin	Reject a company
PUT	/api/admin/companies/<id>/blacklist	Admin	Blacklist company + close drives
GET	/api/admin/students	Admin	List students, search by name/roll/email
PUT	/api/admin/students/<id>/blacklist	Admin	Blacklist a student
GET	/api/admin/drives	Admin	List drives, filter by status
PUT	/api/admin/drives/<id>/approve	Admin	Approve a pending drive

PUT	/api/admin/drives/<id>/reject	Admin	Reject a pending drive
GET	/api/admin/applications	Admin	List all applications
GET	/api/company/dashboard	Company	Own drives + applicant counts
GET/PUT	/api/company/profile	Company	View and update company profile
GET/POST	/api/company/drives	Company	List own drives / create new drive
PUT	/api/company/drives/<id>	Company	Edit a pending drive
PUT	/api/company/drives/<id>/close	Company	Close a drive
GET	/api/company/drives/<id>/applications	Company	List applicants for a drive
PUT	/api/company/applications/<id>/status	Company	Update application status (state machine)
GET	/api/student/dashboard	Student	Stats: drives, applications, placements
GET/PUT	/api/student/profile	Student	View and update student profile
POST	/api/student/profile/resume	Student	Upload resume PDF (max 2MB)
GET	/api/drives	Student	Browse approved drives with filters
GET	/api/drives/<id>	Student	Drive detail page
POST	/api/student/applications	Student	Apply to drive (eligibility + duplicate check)
GET	/api/student/applications	Student	Own application history
POST	/api/exports/applications	Auth	Trigger async CSV export
GET	/api/exports/<id>	Auth	Poll export job status
GET	/api/exports/<id>/download	Auth	Download completed CSV

7. Demo Video and Github

Video Link: <https://drive.google.com/file/d/18c6MLyDJd1hjlru5BL0Cfjk0szPTJK3G/view>

Github Link: <https://github.com/23f2001849/placement-portal-v2-23f2001849>